

五、 实验四 ALU 模块实现

1 实验目的

1. 熟悉 MIPS 指令集中的运算指令，学会对这些指令进行归纳分类。
2. 了解 MIPS 指令结构。
3. 熟悉并掌握 ALU 的原理、功能和设计。
4. 进一步加强运用 verilog 语言进行电路设计的能力。
5. 为后续设计 cpu 的实验打下基础。

2 实验设备

1. 装有 Xilinx Vivado 的计算机一台。
2. LS-CPU-EXB-002 教学系统实验箱一套。

3 实验任务

1. 学习 MIPS 指令集，熟知指令类型，了解指令功能和编码，归纳基础的 ALU 运算指令。
2. 归纳确定自己本次实验中准备实现的 ALU 运算，要求不实现定点乘除指令和浮点运算指令，要求至少实现 5 种 ALU 运算，其中要包含加减运算，其中减法在内部要转换为加法，与加法运算共同调用实验一里自己完成的加法模块去做。
3. 自行设计本次实验的方案，画出结构框图，大致结构框图如图 5.1。图 5.1 中的操作码位数和类型请自行设计，可以设计为独热码（一位有效编码）或二进制编码。比如，设计方案中预定实现 7 种 ALU 运算，则操作码采用独热码，则需 7bit 数据，每位单独指示一种运算；若采用二进制编码，则只用 3bit 数据位即可，但在需 ALU 内部先进行解码，才能确定 ALU 作何种运算。

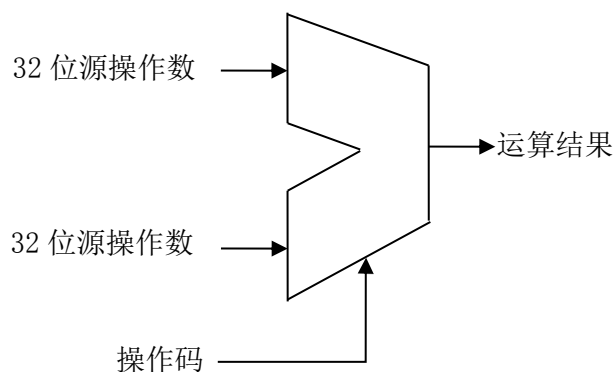


图 5.1 ALU 模块的大致框图

4. 根据设计的实验方案，使用 verilog 编写相应代码。
5. 对编写的代码进行仿真，得到正确的波形图。
6. 将以上设计作为一个单独的模块，设计一个外围模块去调用该模块，见图 5.2。外围模块中需调用封装好的 LCD 触摸屏模块，显示 ALU 的两个源操作数、操作码

和运算结果，并且需要利用触摸功能输入源操作数。操作码可以考虑用 LCD 触摸屏输入，也可以用拨码开关输入。

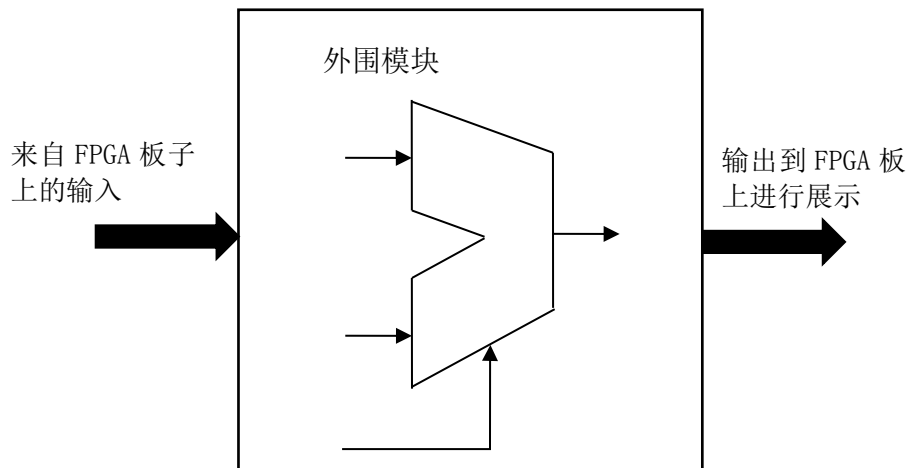


图 5.2 ALU 设计实验的顶层模块大致框图

6. 将编写的代码进行综合布局布线，并下载到试验箱中的 FPGA 板子上进行演示。

4 参考设计

1. 做好预习：

- 1) 熟知指令类型，了解指令功能和编码；
- 2) 归纳基础的 ALU 运算指令，确定自己准备实现的 ALU 运算；
- 3) 设计本次实验的方案，列出准备实现的 ALU 运算和操作码的编码；
- 4) 在课前画好实验方案的设计框图，即补充完善图 5.1；
- 5) 如果对 FPGA 板了解的话，可确定设计中与 FPGA 板上交互的接口，画出包含外围模块的整体设计框图，即补充完善图 5.2。

2. 实验实施：

- 1) 确认 ALU 模块的设计框图的正确性；
- 2) 编写 verilog 代码；
- 3) 对该模块进行仿真，得出正确的波形，截图作为实验报告结果一项的材料；
- 4) 完成调用 ALU 模块的外围模块的设计，并编写代码；
- 5) 对代码进行综合布局布线下载到试验箱里 FPGA 板上，进行上板验证。

3. 实验检查：

- 1) 完成上板验证后，让指导老师或助教进行检查，进行现场演示。先说明自己实现的 ALU 运算类型，按照检查人员的要求，对特定源操作数进行特定运算操作，检查运算结果的正确性，可对演示结果进行拍照作为实验报告结果一项的材料。

4. 实验报告的撰写：

1) 实验结束后，需按照规定的格式完成实验报告的撰写。

5 参考设计

本部分给出的 ALU 实验的参考设计方案中，共实现了 12 种运算，采用独热编码操作码(即 ALU 的控制信号)，对应关系如下：

表 5.1 ALU 的控制信号

控制信号												ALU 操作
11	10	9	8	7	6	5	4	3	2	1	0	
0	0	0	0	0	0	0	0	0	0	0	0	无
1	0	0	0	0	0	0	0	0	0	0	0	加法
0	1	0	0	0	0	0	0	0	0	0	0	减法
0	0	1	0	0	0	0	0	0	0	0	0	有符号比较，小于置位
0	0	0	1	0	0	0	0	0	0	0	0	无符号比较，小于置位
0	0	0	0	1	0	0	0	0	0	0	0	按位与
0	0	0	0	0	1	0	0	0	0	0	0	按位或非
0	0	0	0	0	0	1	0	0	0	0	0	按位或
0	0	0	0	0	0	0	1	0	0	0	0	按位异或
0	0	0	0	0	0	0	0	1	0	0	0	逻辑左移
0	0	0	0	0	0	0	0	0	1	0	0	逻辑右移
0	0	0	0	0	0	0	0	0	0	1	0	算术右移
0	0	0	0	0	0	0	0	0	0	0	1	高位加载

5.1 实验原理

ALU 模块的原理图如下：

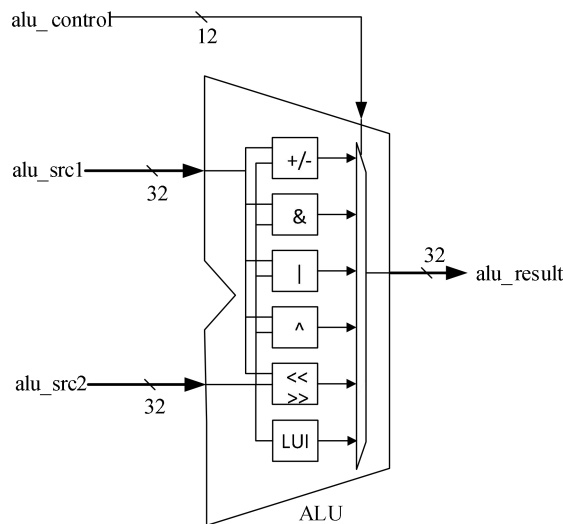


图 5.3 ALU 的原理图

1) 加减运算原理图

加减运算最后都是调用实验一中的加法器做的：

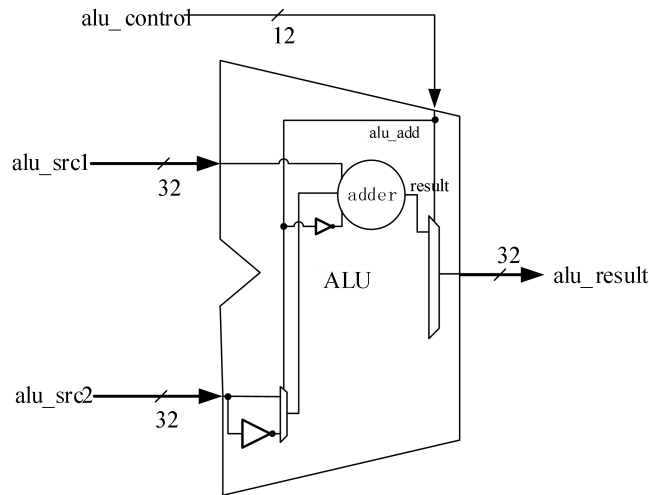


图 5.4 加减法运算的原理图

2) 小于置位运算

对于有符号比较的小于置位，是利用减法结果比较的，其比较的真值表如下：

表 5.2 有符号比较小于置位的真值表

源操作数 1 符号位		源操作数 2 符号位		结果符号位		判断	slt 结果
alu_src1[31]		alu_src2[31]		adder_result[31]			
0	正数	0	正数	0	正数	正>正	0
0	正数	0	正数	1	负数	正<正	1
0	正数	1	负数	X	无关	正>负	0
1	负数	0	正数	X	无关	负<正	1
1	负数	1	负数	0	正数	负>负	0
1	负数	1	负数	1	负数	负<负	1

由表 5.2 可得有符号 32 位比较小于置位运算结果表达式：

$$\text{slt_result} = (\text{alu_src1}[31] \& \sim \text{alu_src2}[31]) \mid (\sim (\text{alu_src1}[31] \wedge \text{alu_src2}[31]) \& \text{adder_result}[31])$$

对于 32 位无符号比较的小于置位，可在其高位前填 0 组合为 33 位正数的比较，即 {1'b0, src1} 和 {1'b0, src2} 的比较，最高位符号位为 0。对比表 5.2 可知，对于正数的比较，只要减法结果的符号位为 1，则表示小于。而 33 位正数相减，其结果的符号位最终可由 32 位加法的 cout+1'b1 得到，如图 5.5。故无符号 32 位比较小于置位运算结果表达式为：sltu_result = ~adder_cout。

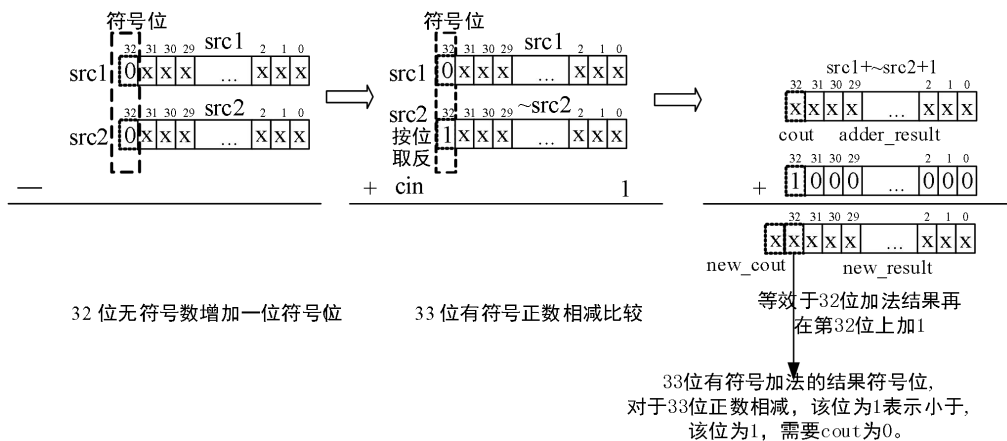


图 5.5 无符号 32 位数比较的原理图

5.2 实验顶层模块框图

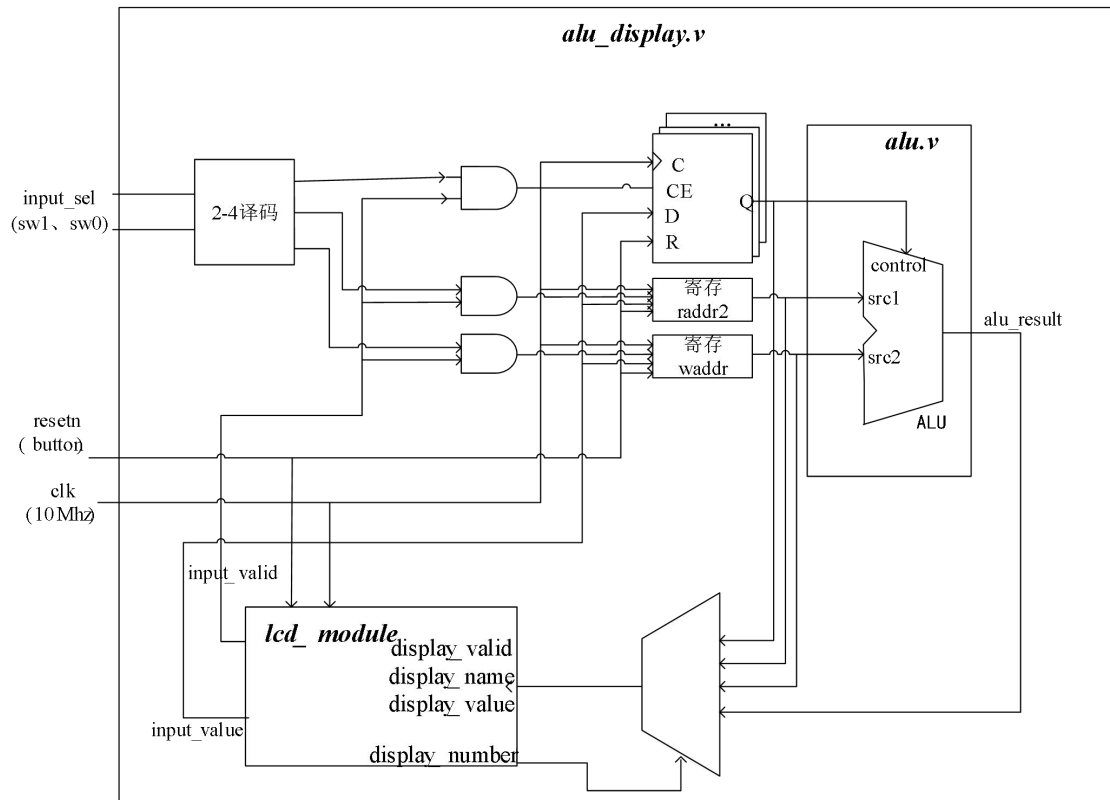


图 5.6 ALU 参考设计的顶层模块框图

5.3 源代码

```

1 //*****
2 // > 文件名: alu_display.v
3 // > 描述 : ALU 显示模块, 调用 FPGA 板上的 I/O 接口和触摸屏
4 // > 作者 : LOONGSON
5 // > 日期 : 2016-04-14
6 //*****
7 module alu_display(
8     //时钟与复位信号
9     input clk,
10    input resetsn,    //后缀"n"代表低电平有效
11
12    //拨码开关, 用于选择输入数
13    input [1:0] input_sel, //00:输入为控制信号(alu_control)
14                        //10:输入为源操作数 1(alu_src1)
15                        //11:输入为源操作数 2(alu_src2)
16
17    //触摸屏相关接口, 不需要更改
18    output lcd_rst,
19    output lcd_cs,
20    output lcd_rs,
21    output lcd_wr,
22    output lcd_rd,
23    inout[15:0] lcd_data_io,
24    output lcd_bl_ctr,
25    inout ct_int,
26    inout ct_sda,

```

```

27     output ct_scl,
28     output ct_rstn
29 );
30 //-----{调用 ALU 模块}begin
31     reg    [11:0] alu_control; // ALU 控制信号
32     reg    [31:0] alu_src1;    // ALU 操作数 1
33     reg    [31:0] alu_src2;    // ALU 操作数 2
34     wire   [31:0] alu_result;   // ALU 结果
35     alu alu_module(
36         .alu_control(alu_control),
37         .alu_src1    (alu_src1   ),
38         .alu_src2    (alu_src2   ),
39         .alu_result  (alu_result )
40     );
41 //-----{调用 ALU 模块}end
42
43 //-----{调用触摸屏模块}begin-----//
44 //-----{实例化触摸屏}begin
45 //此小节不需要更改
46     reg        display_valid;
47     reg [39:0] display_name;
48     reg [31:0] display_value;
49     wire [5 :0] display_number;
50     wire        input_valid;
51     wire [31:0] input_value;
52
53     lcd_module lcd_module(
54         .clk          (clk          ), //10Mhz
55         .resetn       (resetn       ),
56
57         //调用触摸屏的接口
58         .display_valid (display_valid ),
59         .display_name  (display_name  ),
60         .display_value (display_value ),
61         .display_number (display_number),
62         .input_valid   (input_valid   ),
63         .input_value   (input_value   ),
64
65         //lcd 触摸屏相关接口, 不需要更改
66         .lcd_rst       (lcd_rst       ),
67         .lcd_cs        (lcd_cs        ),
68         .lcd_rs        (lcd_rs        ),
69         .lcd_wr        (lcd_wr        ),
70         .lcd_rd        (lcd_rd        ),
71         .lcd_data_io__ (lcd_data_io__ )

```

```
72     .lcd_bl_ctr      (lcd_bl_ctr    ),
73     .ct_int          (ct_int        ),
74     .ct_sda          (ct_sda        ),
75     .ct_scl          (ct_scl        ),
76     .ct_rstn         (ct_rstn       )
77 );
78 //-----{实例化触摸屏}end
79
80 //-----{从触摸屏获取输入}begin
81 //根据实际需要输入的数修改此小节,
82 //建议对每一个数的输入, 编写单独一个 always 块
83 //当 input_sel 为 00 时, 表示输入数控制信号, 即 alu_control
84 always @(posedge clk)
85 begin
86     if (!resetn)
87     begin
88         alu_control <= 12'd0;
89     end
90     else if (input_valid && input_sel==2'b00)
91     begin
92         alu_control <= input_value[11:0];
93     end
94 end
95
96 //当 input_sel 为 10 时, 表示输入数为源操作数 1, 即 alu_src1
97 always @(posedge clk)
98 begin
99     if (!resetn)
100    begin
101        alu_src1 <= 32'd0;
102    end
103    else if (input_valid && input_sel==2'b10)
104    begin
105        alu_src1 <= input_value;
106    end
107 end
108
109 //当 input_sel 为 11 时, 表示输入数为源操作数 2, 即 alu_src2
110 always @(posedge clk)
111 begin
112     if (!resetn)
113     begin
114         alu_src2 <= 32'd0;
115     end
116     else if (input_valid && input_sel==2'b11)
117     begin
118         alu_src2 <= input_value;
119     end
120 end
```

```
121 //-----{从触摸屏获取输入}end
122
123 //-----{输出到触摸屏显示}begin
124 //根据需要显示的数修改此小节,
125 //触摸屏上共有 44 块显示区域, 可显示 44 组 32 位数据
126 //44 块显示区域从 1 开始编号, 编号为 1~44,
127     always @(posedge clk)
128     begin
129         case(display_number)
130             6'd1 :
131                 begin
132                     display_valid <= 1'b1;
133                     display_name  <= "SRC_1";
134                     display_value <= alu_src1;
135                 end
136             6'd2 :
137                 begin
138                     display_valid <= 1'b1;
139                     display_name  <= "SRC_2";
140                     display_value <= alu_src2;
141                 end
142             6'd3 :
143                 begin
144                     display_valid <= 1'b1;
145                     display_name  <= "CONTR";
146                     display_value <= {20'd0, alu_control};
147                 end
148             6'd4 :
149                 begin
150                     display_valid <= 1'b1;
151                     display_name  <= "RESUL";
152                     display_value <= alu_result;
153                 end
154             default :
155                 begin
156                     display_valid <= 1'b0;
157                     display_name  <= 40'd0;
158                     display_value <= 32'd0;
159                 end
160             endcase
161         end
162 //-----{输出到触摸屏显示}end
163 //-----{调用触摸屏模块}end-----//
```


6 实验报告要求

1. 补齐alu.v中代码，包括控制信号、部分逻辑运算（按位与，按位或，按位或非，按位异或，高位加载，逻辑右移6种运算或操作）。
2. 功能仿真：对alu.v进行功能仿真，验证各个算术运算、逻辑运算，每个运算两组激励信号，得出正确波形，将波形和仿真代码放到功能仿真部分。
3. 绑定引脚：和之前要求一样，参照引脚对应关系，截图放到添加约束文件部分。
4. 上板验证，仔细读一下代码，理解控制信号及各个运算如何实现。上板要求每个运算进行两组输入验证，并附简要描述和截图一并放到上板验证部分。